JAVA TYPE SYSTEM STUDY NOTES

ALMAN WASIF

SECURE SOFTWARE

1. Java Variables and Default Values

Key idea

Java automatically gives default values to fields (instance and static variables). Local variables do not get automatic default values and must be initialized before use.

Default values

Type	Default value	Example / meaning
boolean	false	true or false
byte	0	small integer
short	0	integer
int	0	integer
long	0L	large integer
float	0.0f	decimal number
double	0.0	decimal number
char	'\u0000'	null character; appears blank
Reference types (e.g., String)	null	no object reference assigned

Primitive Data Type Sizes

Type	Size (bits)	Size (bytes)	Notes
boolean	JVM-dependent	Not precisely specified	Stores true/false; Java does not define a fixed storage size for boolean.
byte	8	1	Integer
short	16	2	Integer
char	16	2	UTF-16 code unit
int	32	4	Integer
long	64	8	Integer
float	32	4	IEEE 754 floating-point
double	64	8	IEEE 754 floating-point

Note: 1 byte = 8 bits. Java specifies fixed sizes for the numeric primitive types and char; boolean storage size is JVM-dependent.

Example

```
static int count; // default is 0
static boolean active; // default is false
static String name; // default is null
```

Exam tip

Default values apply to fields, not local variables.

2. Type Conversion: Widening and Narrowing

Widening (implicit)

Converting a value to a compatible type that can represent a broader range. It usually happens automatically, for example byte → int, int → long, and long → double. Note: long → float is also allowed automatically, but can lose precision.

Narrowing (explicit)

Converting to a type with a smaller range or precision. It usually requires a cast, for example int → byte or double → long.

Examples

```
int i = 10;
long m = i; // widening: automatic

double d = 5.9;
int whole = (int) d; // narrowing: explicit cast; result is 5
```

Important

Java's common widening path is byte → short → int → long → float → double. char widens to int. This is a useful memory aid, not a complete list of every allowed conversion.

3. Type Casting and Truncation

Casting syntax

(targetType) value

Meaning

The target type is the type you want the value converted into. Casting is an explicit instruction to convert a value.

Example 1

```
int i = 10;
byte b = (byte) i;
System.out.println(b); // 10
```

Example 2: truncation

```
double d = 5.9;
int i = (int) d;
System.out.println(i); // 5
```

Difference

Explicit conversion describes how the conversion is requested (manually with a cast). Truncation describes what happens when a floating-point value is converted to an integer: the fractional part is discarded, not rounded.

4. Assignment Conversion and the = Operator

General form

```
variable = expression;
```

Meaning

```
The assignment operator = stores the value on the right in the variable on the left, provided the types are
compatible.
```

Example

```
int i = 10;
long m = 10000L;
double d = Math.PI;

i = (int) m; // narrowing; cast required
m = i; // widening; automatic
m = (long) d; // narrowing; fractional part discarded
d = m; // widening; automatic
```

Remember

```
The cast goes before the value being converted: b = (byte) i; not b = (int) i when b is a byte.
```

5. Type Promotion in Expressions

Key idea

When different primitive types are used in an arithmetic expression, Java promotes smaller types before doing the calculation.

Rules

Operand(s) in expression	Usual promotion
byte or short or char	promoted to int
If a double operand is present	expression is double
Otherwise, if a float operand is present	expression is float
Otherwise, if a long operand is present	expression is long
Otherwise	expression is int

Example

```
byte a = 10;
short b = 20;
int result = a + b; // result of a + b is int
```

Order to remember

byte → short → int → long → float → double. This is a useful widening order. Even though float has fewer bits than long, long → float is allowed automatically and may lose precision.

6. Constructor Overloading Example

What is a constructor?

A constructor has the same name as its class and no return type. It runs when an object is created.

Overloading

Constructor overloading means having multiple constructors with the same name but different parameter lists.

Given object creation

```
typetest t = new typetest(3.4, 3L, 3);
```

Argument types

3.4 is double; 3L is long (L marks a long literal); 3 is int.

Selected constructor

typetest(double a, double b, double c) is selected because double accepts the first argument exactly, and long → double and int → double are allowed widening conversions.

Output

5 (dbl dbl dbl)

Why not constructor 1 or 3?

Constructor 1 needs a short for the third argument, but int → short is narrowing and is not automatic. Constructor 3 needs a long for the first argument, but double → long is narrowing and is not automatic.

7. Stack and Heap Memory

Stack

Conceptually stores method-call frames and local variables. Each method call has its own frame; frames are removed when calls return.

Heap

Conceptually stores objects and their instance data. Objects created with new are allocated in the heap in the usual Java memory model.

Reference

A local reference variable can be in a stack frame and point to an object on the heap. The reference is not the object itself.

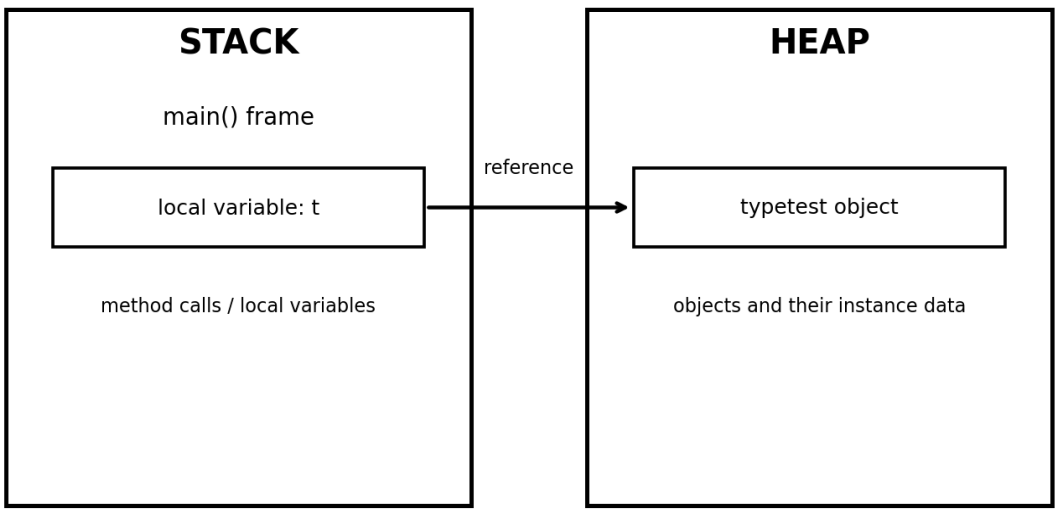


Figure: Simplified relationship between a stack reference and a heap object.

Example

```
typetest t = new typetest(3.4, 3L, 3);
// t is a reference variable in the main() frame (conceptually)
// the typetest object is on the heap
```

Caution

This is a simplified teaching model; JVM optimizations can change the physical implementation.

8. Quick Revision

Remember these

Fields get default values; local variables must be initialized.

Widening is automatic; narrowing usually needs a cast.

double → long is narrowing; long → double is widening (possible precision loss).

Truncation removes the fractional part; it does not round.

byte, short, and char promote to int in arithmetic expressions.

Constructor selection depends on the argument types and allowed conversions.

```
Stack = method calls/local variables (conceptually); heap = objects (conceptually).
```

Self-check Questions

1. What is the default value of a static int field? (Answer: 0)
2. Is double → long widening or narrowing? (Answer: narrowing)
3. What is the result of (int) 5.9? (Answer: 5)
4. What type does byte + short produce? (Answer: int)
5. In the constructor example, which constructor is selected? (Answer: constructor 5: (double, double, double))
6. What does the heap conceptually store? (Answer: objects and their instance data)